

Module 4

Basic Video Compression Techniques - Introduction to Video Compression, Video Compression Based on Motion Compensation, MPEG-1-Video Bitstream, MPEG-2- Supporting Interlaced Video, MPEG-4 - Overview, MPEG-7- Introduction.

Content-Based Retrieval in Digital Libraries- Image Retrieval, CBIRD: A Case Study, Quantifying Search Results, Querying on Videos.

Introduction to Video compression

Video compression is the process of reducing the file size of a video while maintaining an acceptable level of quality. This is essential for efficient storage, transmission, and playback of video content. Since raw video files are extremely large, compression helps to minimize bandwidth and storage requirements without significantly degrading the visual experience.

Why is Video Compression Needed?

1. **Storage Efficiency** – Reduces the amount of space needed to store videos.
2. **Bandwidth Optimization** – Allows smooth streaming over the internet.
3. **Faster Transmission** – Enables quicker uploads, downloads, and sharing.
4. **Better Playback Experience** – Ensures seamless video playback on different devices.

Motion Compensation in Video Compression

Overview

Video compression relies on removing redundant information to reduce storage and bandwidth requirements. One of the key techniques used is **motion compensation**, which eliminates **temporal redundancy** by analyzing similarities between consecutive frames. Instead of storing every frame fully, motion compensation predicts new frames from previous or future frames, significantly reducing the amount of data that needs to be stored or transmitted.

Key Concepts in Motion Compensation

1. Frames and Macroblocks

- **Target Frame:** The current frame that needs to be encoded.
- **Reference Frame(s):** Previously or subsequently encoded frames used to predict the Target Frame.
- **Macroblocks:** Instead of processing individual pixels, frames are divided into small square blocks (typically **16×16 pixels** for luminance and **8×8 pixels** for chrominance) to improve efficiency.

2. Motion Estimation and Motion Vectors

- The encoder searches for a macroblock in the Reference Frame that closely matches a macroblock in the Target Frame.
- The displacement between the corresponding macroblocks is represented by a **Motion Vector (MV)**.
- This search is performed within a **search window** (a limited area around the macroblock's position in the Reference Frame) to reduce computational complexity.

3. Forward and Backward Prediction

- **Forward Prediction:** The Reference Frame is a previous frame, and the Target Frame is predicted using past data.
- **Backward Prediction:** The Reference Frame is a future frame, predicting the Target Frame using upcoming data.
- **Bidirectional Prediction:** Some advanced codecs (e.g., H.264, H.265) use both past and future frames for more accurate prediction.

4. Prediction Error (Residuals)

- Even with motion compensation, there will be slight differences between the predicted and actual macroblock.
- The difference between the predicted macroblock and the actual macroblock is called the **prediction error**.
- The encoder stores only the motion vector and the prediction error instead of the full frame, greatly reducing the file size.

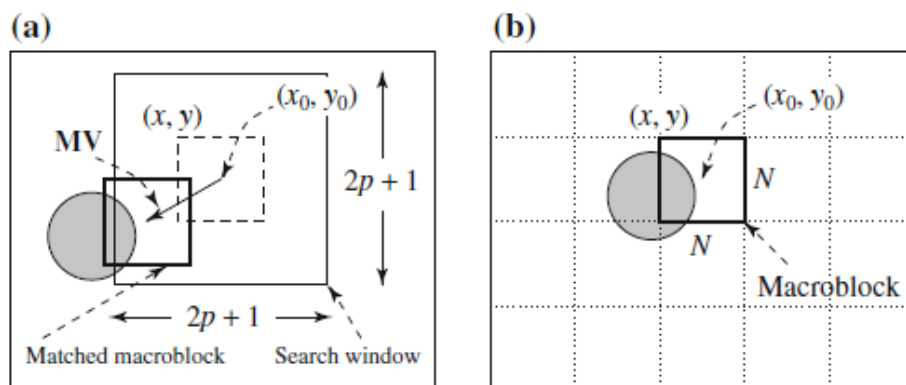


Fig. 10.1 Macroblocks and motion vector in video compression: **a** reference frame; **b** target frame

- Figure 10.1 shows the case of *forward prediction*, in which the Reference frame is taken to be a previous frame

Motion Compensation Process

- **Motion Estimation:** The encoder searches for a matching macroblock in the Reference Frame.
- **Motion Compensation:** The motion vector is recorded, and the predicted macroblock is used in place of the actual block.
- **Residual Encoding:** The difference (prediction error) is encoded separately using transform coding techniques like **Discrete Cosine Transform (DCT)**.

MPEG-1

MPEG-1 is a video and audio compression standard developed by the Moving Picture Experts Group (MPEG) in the early 1990s. It was designed to enable efficient storage and transmission of digital video and audio, particularly for VCD (Video CDs), digital TV broadcasting, and online streaming.

Motion Compensation in MPEG-1

Motion compensation is a fundamental technique in MPEG-1 video compression that reduces **temporal redundancy** by encoding only the changes between consecutive frames instead of storing each frame independently.

Key Steps in Motion Compensation:

1. Each frame is divided into **macroblocks** (e.g., 16×16 pixels).
2. The encoder finds a **matching macroblock** from a reference frame (previous or future).
3. The displacement between the target and reference macroblock is represented by a **motion vector (MV)**.
4. The **difference (prediction error)** between the predicted and actual macroblock is encoded and transmitted.

This technique significantly **reduces data size**, as only the motion vectors and prediction errors need to be stored instead of entire frames.

2. Types of Prediction in MPEG-1 Motion Compensation

MPEG-1 uses three types of frames for motion compensation:

a. Intra-coded frames (I-frames)

- Encoded independently without referencing other frames.
- Uses spatial compression techniques like DCT.
- Serves as keyframes that provide a full picture.
- Larger in size compared to other frame types.

b. Predictive-coded frames (P-frames)

- Use forward prediction from the nearest previous I-frame or P-frame.
- Contain motion vectors and prediction errors, reducing storage size.
- More efficient than I-frames but still contain some residual data.

c. Bidirectionally predictive-coded frames (B-frames)

- Use both forward and backward prediction from surrounding frames (previous and future).
- Motion vectors are computed from two reference frames and averaged.
- More efficient compression but requires buffering since future frames must be available.
- B-frames never serve as reference frames.

3. Forward and Bidirectional Prediction

- **Forward Prediction (P-frames):** The current macroblock is predicted using a previous reference frame. If an exact match isn't found, only the difference is stored.
- **Backward Prediction:** The macroblock is predicted from a future reference frame.
- **Bidirectional Prediction (B-frames):** The encoder searches for the best match in both previous and future frames. If two good matches are found, the average is used to improve accuracy (as shown in **Fig. 11.2**).

This technique allows higher compression efficiency while maintaining good visual quality.

4. Encoding Process in MPEG-1 (As seen in Fig. 11.2)

1. **Motion Estimation:** The encoder searches for the best matching macroblock in reference frames.
2. **Motion Compensation:** The motion vector and prediction error are calculated.

3. **Discrete Cosine Transform (DCT):** The prediction error is transformed into frequency components to remove redundant information.
4. **Quantization:** Less important data is discarded to improve compression.
5. **Entropy Coding:** The final compressed data is stored as a bitstream (e.g., **0011101...**).

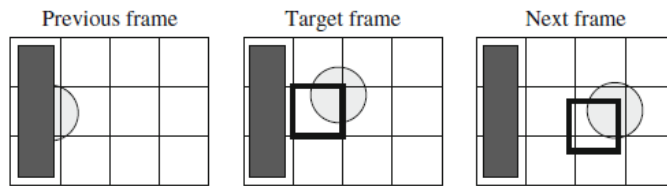


Fig. 11.1 The need for bidirectional search

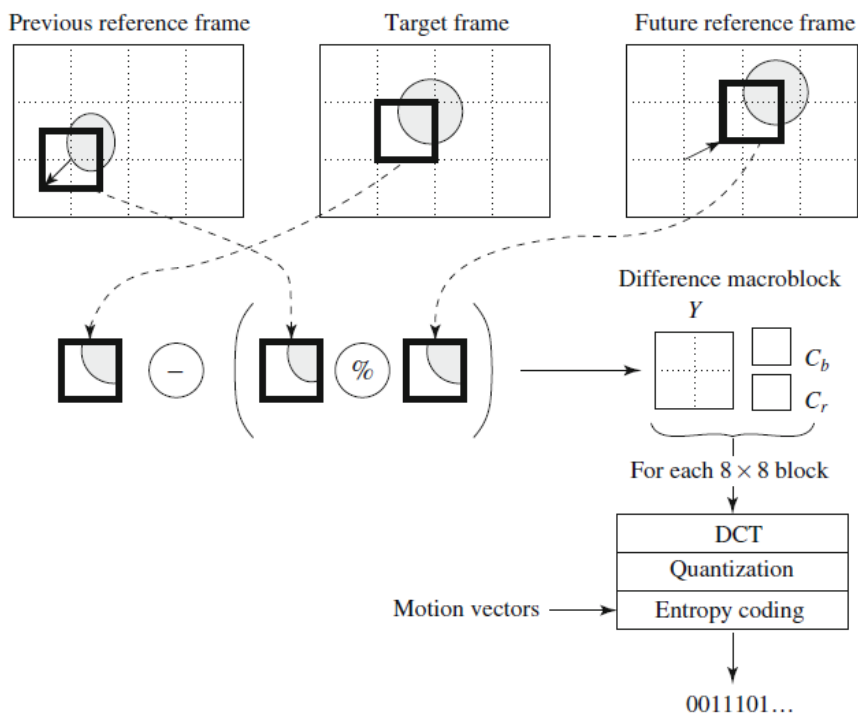


Fig. 11.2 B-frame coding based on bidirectional motion compensation

5. Frame Sequence in MPEG-1 (Fig. 11.3 Reference)

- MPEG-1 defines **two parameters** to control frame sequences:
 - **M** = The number of frames between a P-frame and its preceding I-frame.
 - **N** = The number of frames between two consecutive I-frames.
- Example sequence: **I → B → B → P → B → B → P → I**, where **M = 3**, **N = 9**.
- If **M = 1**, no B-frames are used, resulting in lower compression efficiency.

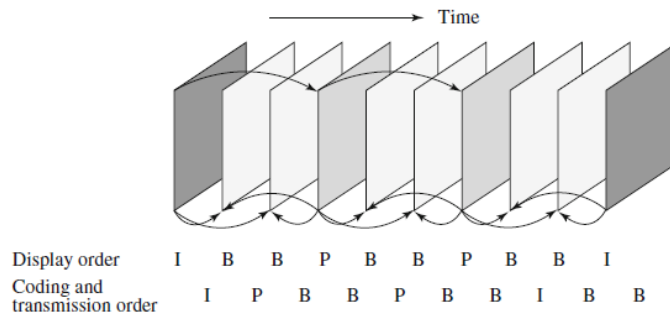


Fig. 11.3 MPEG frame sequence

MPEG-1 Video Bitstream

1. **Sequence layer** A video sequence consists of one or more groups of pictures (GOPs). It always starts with a sequence header. The header contains information about the picture, such as *horizontal_size* and *vertical_size*, *pixel_aspect_ratio*, *frame_rate*, *bit_rate*, *buffer_size*, *quantization_matrix*, and so on.

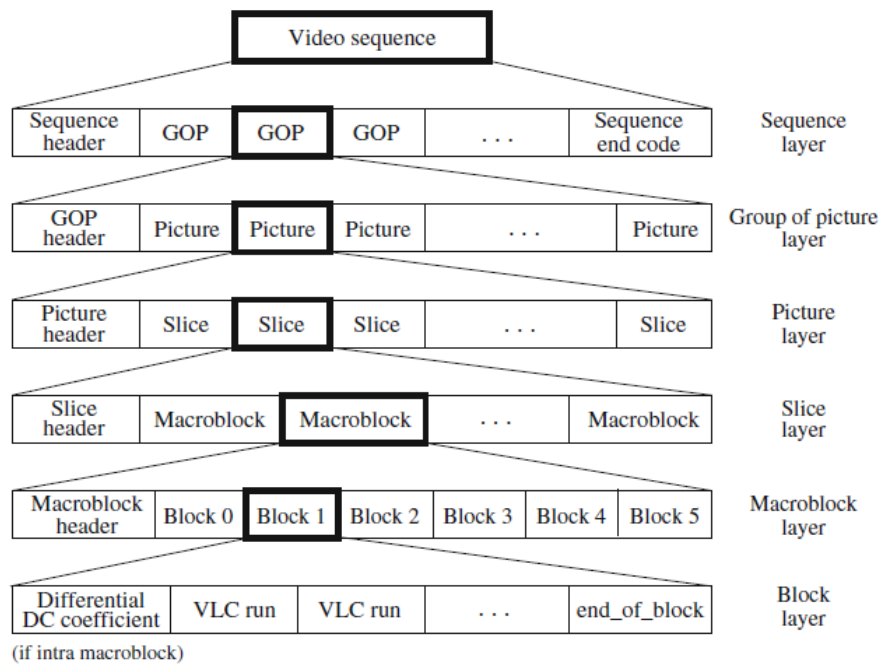


Fig. 11.5 Layers of MPEG-1 video bitstream

2. **GOPs layer** A GOP contains one or more pictures, one of which must be an I-picture. The GOP header contains information such as *time_code* to indicate hour-minute-second-frame from the start of the sequence.
3. **Picture layer** The three common MPEG-1 picture types are *I-picture* (intra-coding), *P-picture* (predictive coding), and *B-picture* (Bidirectional predictive coding), as discussed above. There is also an uncommon type, *D-picture* (DC coded), in which only DC coefficients are retained. MPEG-1 does not allow mixing D-pictures with other types, which makes D-pictures impractical.
4. **Slice layer** As mentioned earlier, MPEG-1 introduced the slice notion for bitrate control and for recovery and synchronization after lost or corrupted bits. Slices may have variable numbers of macroblocks in a single picture. The length and position of each slice are specified in the header.
5. **Macroblock layer** Each macroblock consists of four *Y* blocks, one *C_b* block, and one *C_r* block. All blocks are 8×8 .
6. **Block layer** If the blocks are intracoded, the differential DC coefficient (DPCM of DCs, as in JPEG) is sent first, followed by variable-length codes (VLC), for AC coefficients. Otherwise, DC and AC coefficients are both coded using the variable-length codes.

MPEG-2:

Overview:

- Developed in 1990 to improve upon MPEG-1, supporting higher bitrates (4 Mbps+) and better video quality.
- Designed for digital television (DTV), DVDs, and HDTV broadcasting.
- Introduced interlaced video support and multiple profiles & levels for different applications.
- Includes more efficient motion compensation techniques.

Key Features of MPEG-2 Compression:

1. Improved Motion Compensation:

- Uses the same I, P, and B-frames as MPEG-1 but with better motion estimation techniques.
- **Supports larger search windows** for motion vectors, allowing better tracking of moving objects.
- **Reduces blocking artifacts**, leading to smoother video playback.

2. Interlaced Video Support

Unlike MPEG-1 (which only supports progressive video), MPEG-2 introduces interlaced video, essential for broadcast television.

- Interlaced scanning divides a frame into two fields:
 - Top-field (odd-numbered scan lines)
 - Bottom-field (even-numbered scan lines)
- Frame-picture vs. Field-picture:

- A Frame-picture combines both fields into one full image.
- A Field-picture treats each field as a separate image, reducing motion blur in fast-moving scenes.

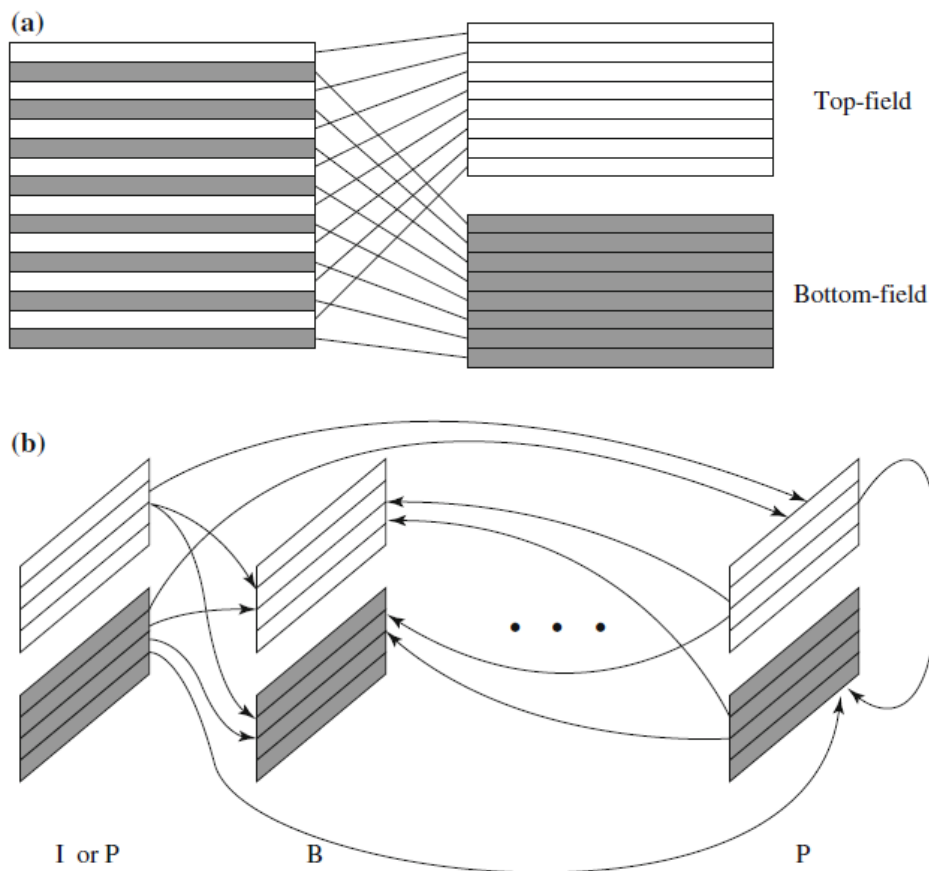


Fig. 11.6 Field pictures and field prediction for field pictures in MPEG-2: **a** frame-picture versus field pictures; **b** field prediction for field pictures

Five Modes of Predictions (MPEG-2)

MPEG-2 supports **frame prediction** and **field prediction** with 5 different modes for different video scenarios.

1. Frame Prediction for Frame-Pictures

- Same as MPEG-1 method.
- Used in both P-frames and B-frames.
- Works well for slow/moderate motion videos.

2. Field Prediction for Field Pictures

- Uses 16×16 macroblocks.
- Predictions are made from previously encoded **fields**.
- **Top-field** macroblocks predict from top or bottom fields of earlier frames.
- **Bottom-field** macroblocks do the same.
- In B-frames, both forward and backward prediction is used.

- No strict rule to match top/bottom fields from reference frames.

3. Field Prediction for Frame-Pictures

- Splits each 16×16 macroblock into two 16×8 blocks (one per field).
- Each 16×8 block is predicted separately.
- Used in both P and B frames.
- Up to 2 motion vectors for P-frames and 4 for B-frames.

4. 16×8 Motion Compensation for Field Pictures

- Each 16×16 macroblock split into two 16×8 halves (top 8 rows + bottom 8 rows).
- Each half predicted independently.
- Results in:
 - 2 motion vectors for P-frames.
 - Up to 4 motion vectors for B-frames.
- Suitable for **rapid and irregular motion**.

5. Dual-Prime for P-Pictures

- Works with both frame and field pictures.
- Uses primary and calculated secondary motion vectors.
- Helps improve prediction by considering spacing between lines (interlacing).
- Good for encoding efficiency with P-frames.

Feature	MPEG-1	MPEG-2
Error Resilience	Not very robust to bit errors	Improved error resilience using Transport Stream with 188-byte fixed packets and enhanced headers
Stream Types	Systems stream only	Program Stream (like MPEG-1) and Transport Stream (for noisy networks, multiple programs, better sync)
Chroma Subsampling	Only supports 4:2:0	Supports 4:2:0, 4:2:2, and 4:4:4 for better color quality
Quantization Scaling	Linear quantization scale (scale = i)	Supports both linear and nonlinear quantization (scale $\approx \alpha i$)
Slice Structure	Slices can cross macroblock row boundaries; entire image can be one slice	Slices must start and end in same row ; max length = one row of macroblocks
Video Format Flexibility	Limited to lower resolutions	Supports resolutions up to 16k × 16k , used in DVD, ATV, HDTV
Encoding Specification	Simpler	More complex bitstream and decoder syntax; still leaves encoder open to improvements

MPEG-4

MPEG-4 is a **multimedia coding standard** developed by the Moving Picture Experts Group (MPEG), designed to **encode, transmit, and decode** audio-visual data efficiently while supporting **interactivity, scalability, and flexibility** across a wide range of platforms (like the Internet, mobile devices, and interactive TV).

MPEG-4 introduces **object-based coding** using **media objects**.

What are media objects?

- Can be **visual or audio**, either **natural** (captured by a camera) or **synthetic** (generated by a computer).
- Example: A banana, drink, or animated icon.

Video Object Planes (VOPs)

- MPEG-4 video frames are segmented into **individual objects**, each coded separately.
- VOPs can be **added, deleted, moved, scaled, or rotated** individually.
- Enables **content-based manipulation** (see Fig. 11.11).

3. Scene Composition

- MPEG-4 composes scenes from multiple VOPs.
- Allows mixing of live video, animation, graphics, and text.
- Similar to layers in image editing.

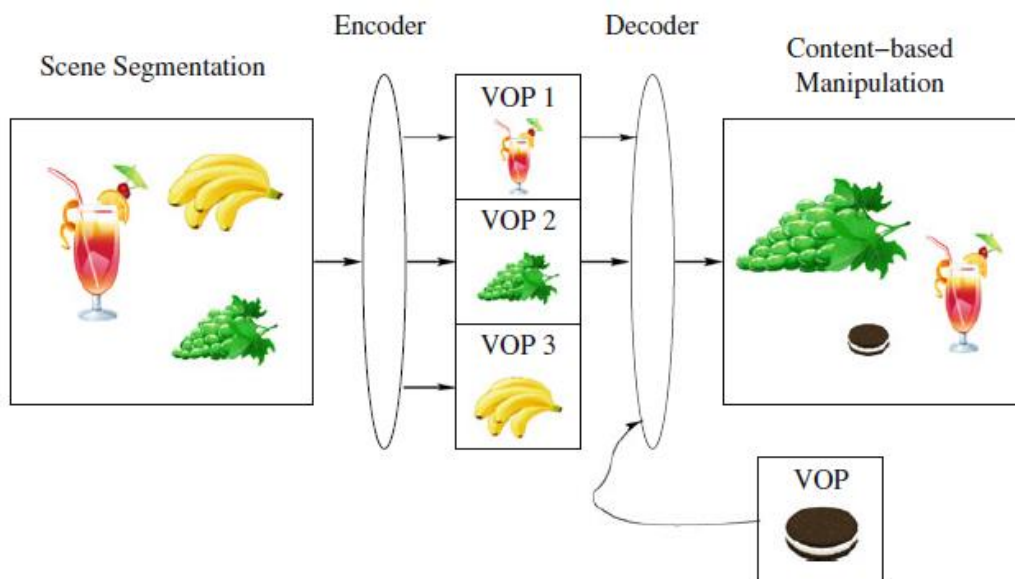


Fig. 11.11 Composition and manipulation of MPEG-4 videos (VOP = Video object plane)

❑ MPEG-4 supports **user interaction** with visual objects, such as clicking, zooming, or rotating video elements in real-time.

Fig. 11.12 Comparison of interactivity in MPEG standards: **a** reference models in MPEG-1 and 2 (interaction in *dashed lines* supported only by MPEG-2); **b** MPEG-4 reference model

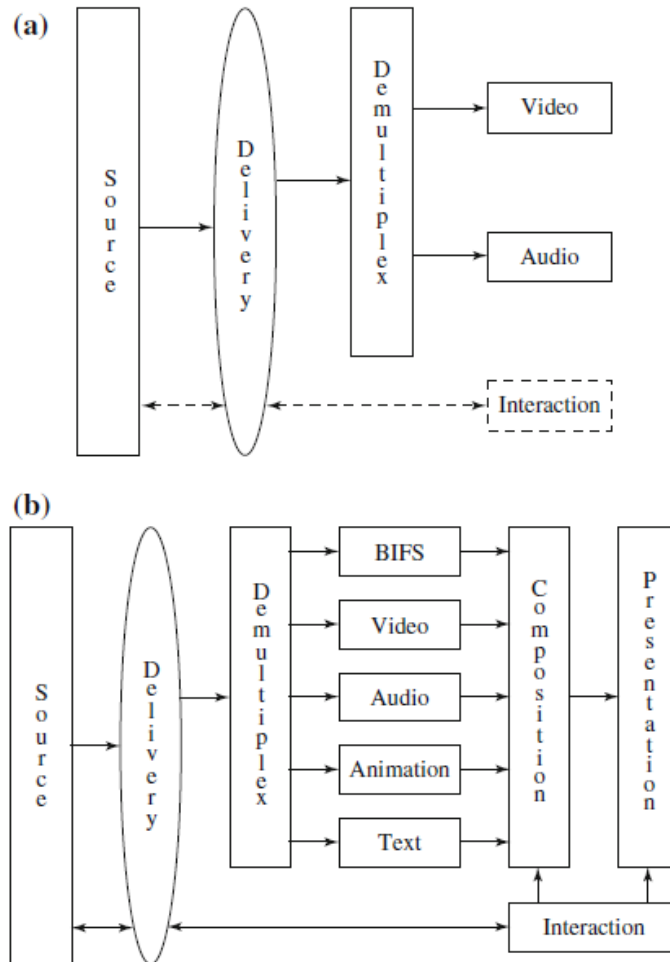


Fig. 11.12(a): MPEG-1 and MPEG-2 allow only limited interaction (in dashed lines for MPEG-2).

Fig. 11.12(b): MPEG-4 fully supports interaction at various stages—composition, presentation, and content access.

Binary Format for Scenes (BIFS)

- BIFS is used to **compose and describe** scenes made from multiple objects.
- Based on the **VRML** (Virtual Reality Modeling Language), but enhanced to include **timing, synchronization, and spatial/temporal relationships**.
- Allows for scene graphs where each node describes an audiovisual object and its behavior.
- The hierarchical structure of MPEG-4 visual bitstreams is very different from
- that of MPEG-1 and 2 in that it is very much video-object-oriented. Figure 11.13
- illustrates five levels of the hierarchical description of a scene in MPEG-4 visual bitstreams.

bitstreams. In general, each *Video-object Sequence* (VS) will have one or more *Video Objects* (VOs), each VO will have one or more *Video Object Layers* (VOLs), and so on. Syntactically, all five levels have a unique start code in the bitstream, to enable random access.

1. **Video-object Sequence (VS).** VS delivers the complete MPEG-4 visual scene, which may contain 2D or 3D natural or synthetic objects.
2. **Video Object (VO).** VO is a particular object in the scene, which can be of arbitrary (nonrectangular) shape, corresponding to an object or background of the scene.
3. **Video Object Layer (VOL).** VOL facilitates a way to support (multilayered) scalable coding. A VO can have multiple VOLs under scalable coding or a single VOL under nonscalable coding. As a special case, MPEG-4 also supports a special type of VOL with a shorter header. This provides bitstream compatibility with the baseline H.263 [12].
4. **Group of Video Object Planes (GOV).** GOV groups video object planes. It is an optional level.
5. **Video Object Plane (VOP).** A VOP is a snapshot of a VO at a particular moment, reflecting the VO's shape, texture, and motion parameters at that instant. In general, a VOP is an image of arbitrary shape. A degenerate case in MPEG-4 video coding occurs when the entire rectangular video frame is treated as a VOP. In this case, it is equivalent to MPEG-1 and 2. MPEG-4 allows overlapped VOPs—that is, a VOP can partially occlude another VOP in a scene.

Video-object Sequence (VS)
Video Object (VO)
Video Object Layer (VOL)
Group of VOPs (GOV)
Video Object Plane (VOP)

Fig. 11.13 Video-object-oriented hierarchical description of a scene in MPEG-4 visual bitstreams

MPEG-7 (Multimedia Content Description Interface)

Introduction

- **MPEG-7** is a standard developed by the MPEG group (following MPEG-1, -2, and -4) with the goal of **efficient content-based multimedia retrieval**.
- Unlike previous MPEG standards (which focused on compression and transmission), MPEG-7 addresses **multimedia content description**, enabling fast and accurate search and retrieval of audiovisual data.

Main Objective

- To facilitate **audiovisual content-based retrieval**, including images, video, audio, 3D models, and more.

- Useful in **digital libraries, multimedia databases, content management systems, and search engines.**
- Supports both **content creation** and **content consumption.**

Standard Structure

The MPEG-7 standard is composed of several **Parts**:

1. **Systems** – Describes encoding/decoding and synchronization methods.
2. **Description Definition Language (DDL)** – A language to define Descriptors and Description Schemes (uses XML Schema).
3. **Visual** – For visual descriptors (e.g., color, shape).
4. **Audio** – For audio descriptors (e.g., pitch, timbre).
5. **Multimedia Description Schemes (MDS)** – High-level semantic description.
6. **Reference Software**
7. **Conformance and Testing**

Data Representation

- MPEG-7 data can be represented in:
 - **Textual format** (based on XML)
 - **Binary format** (BiM: Binary format for MPEG-7)
 - **Both textual and binary, with a bidirectional lossless mapping** between them

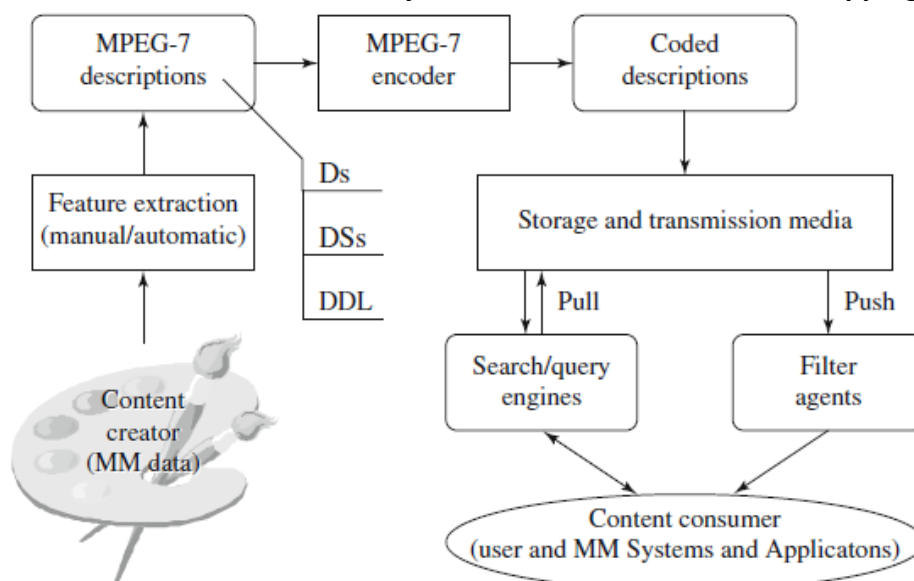


Fig. 11.27 Possible applications using MPEG-7

Term	Description
Feature	A characteristic of the multimedia data (e.g., color, motion, shape)
Descriptor (D)	A formal definition of a feature (syntax + semantics)
Description Scheme (DS)	Specifies structure and relationships between descriptors
Description	A set of instantiated Ds and DSs to represent structural and conceptual multimedia content

Term	Description
Description Language (DDL)	Definition The syntactic language (based on XML Schema) to define and combine Ds and DSs - Enables efficient storage, transmission, and retrieval of multimedia based on content semantics. - Supports both push and pull multimedia interactions: Push: Content is sent to terminals (e.g., streaming platforms) Pull: Users query/search content (e.g., Google Images)

◆ MPEG-7 Workflow Overview

1. **Feature Extraction** – Identify relevant characteristics from the content.
2. **Instantiation of MPEG-7 Descriptions** – Create descriptions using Descriptors and Description Schemes.
3. **Encoding and Transmission** – Use MPEG-7 encoder to transmit descriptions.
4. **Query/Searching/Matching** – Retrieve media using semantic content.

Descriptor (D), Description Scheme (DS), and Description Definition Language (DDL) in MPEG-7

In MPEG-7, **Descriptors (Ds)** are used to describe both **low-level features** (such as color, texture, shape, and motion) and **high-level features** (like events or abstract semantic concepts). Although the standard doesn't define methods for feature extraction, it supports descriptors that are carefully selected based on performance, efficiency, and size. Low-level visual descriptors include **color descriptors** (like color space, scalable color, color layout, and dominant colors), **texture descriptors** (like homogeneous texture using Gabor filters, texture browsing, and edge histograms), and **shape descriptors** (both region-based and contour-based using ART and CSS techniques, respectively). MPEG-7 also provides **motion descriptors** (like camera motion, object trajectory, parametric motion, and motion activity), **localization descriptors** for identifying regions in images and videos, and even support for **face recognition** using vector projection methods.

Description Schemes (DSs) define the structure and relationships between descriptors and organize them to describe complex multimedia content. DSs cover a wide range of applications such as basic elements (datatypes, constructs, schema tools), **content management** (media info, creation, usage), **content description** (structural segments, conceptual elements), **navigation and access** (summaries, view partitions, content variations), **content organization** (collections and models), and **user interaction** (user preferences). For instance, the Segment DS helps describe structural elements of content like video or image regions and their relationships, while Concept DSs support high-level annotations such as events or abstract themes.

The **Description Definition Language (DDL)** in MPEG-7 is based on **XML Schema** from W3C, enhanced with MPEG-7 specific extensions. It defines the syntax to express and combine Ds and DSs.

The DDL includes schema structure components (such as type definitions and element declarations), datatype components (primitive and derived types), and MPEG-7 specific features like support for array and matrix data types, multiple media formats, enumerated codes for countries and currencies, and tools for Intellectual Property Management and Protection (IPMP). Altogether, D, DS, and DDL form the foundation of MPEG-7's capability to describe, organize, and enable the retrieval of multimedia content.

CBIR (Content-Based Image Retrieval):

Content-Based Image Retrieval (CBIR) is a technique used to search and retrieve images from a large database based on the **actual visual content** of the images rather than relying solely on metadata or accompanying text. CBIR analyzes features such as **color, texture, shape, and spatial layout** to find similarities between images. For example, if a user searches for images with deep blue skies or orange sunsets, CBIR can use precomputed visual statistics to return relevant results. Unlike traditional text-based image search, which may struggle with abstract or artistic works like Hieronymus Bosch's *The Garden of Delights*, CBIR provides a way to find and classify images based on what they visually depict. While it may not fully capture the **semantics** or deeper meanings in complex artwork, CBIR represents a shift towards **using multimedia content itself** as a primary basis for retrieval, either alone or in combination with text-based methods.

Modern image retrieval systems have begun to address the **semantic content** of images, combining **textual metadata** with **visual features** to enhance accuracy. However, many techniques still focus solely on **image features** like **color histograms, color layout, and texture** to retrieve similar images without relying on accompanying text. These features help identify visual patterns, such as bright reds in a Santa image or edge-based textures, and are particularly useful in fields like art, fashion, GIS, and trademark databases.

Despite progress, significant challenges remain. The **sensory gap** refers to the difference between real-world objects and their computational representations, while the **semantic gap** highlights the disconnect between visual data and the **human interpretation** of image meaning. These gaps make it difficult for machines to fully understand image semantics, particularly in complex or abstract scenes. Advanced searches, like locating specific objects within images or 3D shape retrieval, are even more demanding and often require iterative **search-by-association** methods for refinement.

C-BIRD : CASE STUDY

C-BIRD (Content-Based Image Retrieval from Digital libraries) is an early image retrieval system designed to search a database of ~5,000 images using various **image-based features**. It supports tools like **text annotations, color histograms, illumination-invariant histograms, color/texture layout, and object-based search**.

A key feature is the **color histogram**, which records pixel color distributions in red, green, and blue (RGB) channels. Instead of fine-grained 256^3 bins, C-BIRD uses a **quantized version** with **3 bits for red**

and green, and 2 bits for blue to reduce storage and enhance similarity matching. These histograms are precomputed for all images and compared with user queries to find visually similar results. C-BIRD also introduces unique functionalities like **Search by Illumination Invariance** and **Feature Localization**, enhancing robustness to lighting changes and object-level search.

Color Histogram

A **color histogram** is a type of **global image feature** that represents the distribution of colors in an image. It **counts the number of pixels** for each color value across the image, regardless of where those pixels are located.

This makes color histograms:

- **Rotation-invariant**
- **Robust to occlusions**
- **Efficient** for capturing overall color content

In **RGB color space**, each pixel is represented by three values (Red, Green, Blue), typically with **8-bit depth**, allowing 256 values per channel. A full histogram would have $256 \times 256 \times 256 = 16,777,216$ bins, which is computationally expensive.

A color histogram counts pixels with a given pixel value in red, green, and blue (RGB). For example, in pseudocode, for images with 8-bit values in each of R, G, B, we can fill a histogram that has 256³ bins:

```
int hist[256][256][256]; // reset to 0
//image is an appropriate struct
//with byte fields red,green,blue
for i=0..(MAX_Y-1)
for j=0..(MAX_X-1)
{
R = image[i][j].red;
G = image[i][j].green;
B = image[i][j].blue;
hist[R][G][B]++;
}
```

Key Functionalities:

1. Color Histogram:

- Global feature that counts RGB pixel values.
- Quantized bins reduce storage and emphasize similarity.
- Used to match feature vectors of query and database images.

2. Texture Layout:

- Users specify desired texture layout using a grid.
- Textures classified by **directionality** (0°, 45°, 90°, 135°) and **density** (low/medium/high).
- Useful in identifying texture-rich areas in images.

3. Texture Analysis:

- Based on **Tamura's features**: directionality, granularity, and repetitiveness.
- Uses **Sobel edge detection** to extract edge maps.
- Constructs 2D texture histograms (edge direction vs. separation), then reduces and normalizes them.

4. Illumination-Invariant Search:

- Handles lighting variations by normalizing RGB channels.
- Creates chromaticity histograms independent of brightness.
- Histograms are compressed (e.g., wavelet, DCT) and compared using Euclidean distance in compressed form.

5. Search by Object Model:

- Most powerful feature—enables object-based search.
- User selects object via tools (e.g., brush, ellipse, magic wand).
- Matching process includes:
 - Chromaticity histogram screening
 - Pose estimation (scale, rotation, translation)
 - Texture matching
 - Shape verification via **Generalized Hough Transform**

20.4 Quantifying Search Results

Generally speaking, some simple expression of the performance of image search engines is desirable. In information retrieval, *Precision* is the percentage of relevant documents retrieved compared to the number of all the documents retrieved, and *Recall* is the percentage of relevant documents retrieved out of all relevant documents. Recall and Precision are widely used for reporting retrieval performance for image retrieval systems as well. However, these measures are affected by the database size

and the amount of similar information in the database. Also, they do not consider fuzzy matching or search result ordering.

In equation form, these quantities are defined as:

$$\begin{aligned} \text{Precision} &= \frac{\text{Relevant images returned}}{\text{All retrieved images}} \\ \text{Recall} &= \frac{\text{Relevant images returned}}{\text{All relevant images}} \end{aligned} \quad (20.6)$$

Alternatively, they may also be written as:

$$\begin{aligned} \text{Precision} &= \frac{TP}{TP + FP} \\ \text{Recall} &= \frac{TP}{TP + FN} \end{aligned} \quad (20.7)$$

where TP (True Positives) is the number of relevant images returned, FP (False Positives) is the number of irrelevant images returned, and FN (False Negatives) is the number of relevant images not returned.

In general, the more we relax thresholds and allow more images to be returned, the smaller the Precision, but the larger the Recall; and vice versa. Apparently, it is not quite meaningful to talk about either the Precision or Recall number by itself. Instead, they can be combined to provide a good measure, e.g., Precision when Recall is at 50 %, Recall when Precision is at 90 %, etc.

When multiple queries are involved, the numbers of the Precision and Recall values will again increase. To measure the overall performance of a CBIR system, the most common way is to summarize these values into a single value, i.e., the *Mean Average Precision (MAP)*. The Average Precision (AP) of a single query q is defined as:

$$AP(q) = \frac{1}{N_R} \sum_{n=1}^{N_R} \text{Precision}(n), \quad (20.8)$$

where $\text{Precision}(n)$ is the Precision value after the n th relevant image was retrieved, and N_R is the total number of relevant images. The MAP is the mean of Average Precisions over all query images:

$$MAP = \frac{1}{N_Q} \sum_{q \in Q} AP(q), \quad (20.9)$$

where Q is the query image set, and N_Q is its size. The MAP has the advantage of reflecting both Precision and Recall oriented aspects and is sensitive to the entire

QUERYING On Videos

Querying on videos generally refers to the process of searching for specific video content within large collections, often using certain features or metadata to find relevant results. This can be done by querying based on visual content, audio content, textual descriptions, or a combination of these.

- When querying on videos, the focus is usually on improving the retrieval process based on the user's needs, ensuring that the system can return the most relevant videos based on the query parameters.
- The first place to start, then, would be dividing the video into shots, where each shot consists roughly of the video frames between the on and off clicks of the Record button. However, transitions are often placed between shots—fade-in, fade-out, dissolve, wipe, and so on—so detection of shot boundaries may not be so simple as for abrupt changes.
- Generally, since we are dealing with digital video, if at all possible we would like to avoid uncompressing MPEG files, say, to speed throughput. Therefore, researchers try to work on the compressed video. A simple approach to this idea is to uncompress just enough to recover the DC term, generating a thumbnail 64 times smaller than the original. Since we must consider P- and B-frames as well as I-frames, even generating a good approximation of the best DC image is itself a complicated problem.
- Shots are grouped into scenes. A scene is a collection of shots that belong together and that are contiguous in time. Even higher-level semantics exist in so-called “film grammar”. Semantic information such as the basic elements of the story may be obtainable.
-